PROJECT REPORT

SMART WASTE MANAGEMENT SYSTEM

A MERN-Based Web Application for Waste Reporting and Collection Tracking

Submitted By:

M.SAI CHARAN -25MVCSMR0058
P.ROHITH --25MVCSMR0065
B.STALIN -25MVCSMR0015
V.BHARATH REDDY-25MVCSMR0091

Submitted to

Department of Computer Science and Engineering
Artificial Intelligence and Machine Learning
Mallareddy Vishwavidyapeeth Technical Campus

Academic Year: 2026–2027

TABLE OF CONTENTS

1. Abstract

2. Introduction

3. Problem Statement, Objectives and Scope

4. Features and Functional Requirements

5. Algorithms and Methodology

6. Technologies and Implementation

7. Results, Testing and Performance

8. Conclusion, Future Scope and References

Appendix A. Proposed Project Modules

Appendix B. Proposed Project Workflow

Appendix C. Glossary

Note: Page numbers may change slightly depending on the version of Microsoft Word and local font settings.

1. ABSTRACT

Waste management is an important public service that supports cleanliness, environmental protection, and community health. In many places, waste issues such as overflowing bins, roadside dumping, and accumulated household garbage are reported through manual or informal communication. These methods may make it difficult to maintain consistent records, monitor complaint status, and identify locations that repeatedly require attention.

The Smart Waste Management System is proposed as a web-based application developed using the MERN stack: MongoDB, Express.js, React.js, and Node.js. It provides a digital channel for users to report garbage-related problems and follow the progress of submitted reports. A report can include a title, waste category, location, description, and an image. The backend validates the submitted information and stores report data in MongoDB. Authorized administrators can review records and update their status as work progresses.

The intended status workflow contains three principal stages: Pending, In Progress, and Resolved. A dashboard can summarize the reports and provide search and filtering options. Image storage can be connected through a service such as Cloudinary, while location details can initially be entered manually and later enhanced with GPS or map integration.

The project aims to improve the organization and transparency of waste reporting rather than replace the physical work of waste collection. Its effectiveness depends on correct implementation, secure access controls, reliable data, testing, and coordination with responsible collection teams. The project also provides a foundation for future additions such as notifications, map visualization, duplicate detection, analytics, and AI-assisted waste classification.

Keywords: Smart Waste Management, MERN Stack, MongoDB, React.js, Express.js, Node.js, Waste Reporting, Collection Tracking, Environmental Sustainability.

2. INTRODUCTION

2.1 Background

Waste is generated by households, markets, offices, educational institutions, construction activities, and commercial establishments. If it is not collected and handled appropriately, it can affect the appearance of public spaces, contribute to pollution, and create conditions that are unpleasant or unhealthy for communities. Waste management therefore requires coordination among residents, collection workers, service providers, and local authorities.

A common challenge is not only the physical collection of garbage but also the reporting and tracking of problems. Residents may notice waste accumulation but may not know the correct channel for reporting it. Even after a complaint is submitted, the person who reported it may have little visibility into whether the issue has been assigned, addressed, or closed.

2.2 Digital Approach

A web-based reporting system can provide a common place to submit and organize reports. Users can provide structured information instead of relying entirely on unstructured messages. Administrators can review records in one interface and update their status. A digital record also makes it possible to search past reports, identify recurring problem areas, and summarize workload.

2.3 Project Overview

The Smart Waste Management System focuses on reporting and collection tracking. React.js provides the interface, Express.js defines HTTP routes, Node.js runs the backend application, and MongoDB stores report records. If image uploads are included, an external image-storage service can host the files while MongoDB stores the returned URL.

2.4 Need for the Project

- Provide an accessible channel for reporting waste problems.
- Keep records in a centralized and searchable database.
- Make report status visible to authorized users.
- Help administrators prioritize unresolved reports.
- Support basic summaries of waste type and report status.
- Create a base for future smart-city and sustainability features.

3. PROBLEM STATEMENT, OBJECTIVES AND SCOPE

3.1 Problem Statement

Garbage accumulation and improper disposal are often communicated through manual or informal channels. These approaches may produce fragmented information, make follow-up difficult, and provide limited visibility into the resolution of a complaint. A centralized web application is needed so that citizens can submit structured waste reports and authorized waste-management personnel can review, update, and track those reports.

3.2 Objectives

1. Enable users to submit a report with location, waste type, description, and an image.
2. Track the progress of reports through Pending, In Progress, and Resolved statuses.
3. Store report information in MongoDB for retrieval and management.
4. Provide authorized administrators with tools to review and update reports.
5. Record location information to help identify areas that need cleaning.
6. Support image uploads as visual context for reported problems.
7. Provide search and filtering for efficient report management.
8. Build a responsive and understandable interface using React.js.
9. Implement REST APIs using Node.js and Express.js.
10. Encourage responsible waste disposal and community participation.

3.3 Project Goals

The immediate goal is to demonstrate a working end-to-end reporting workflow: a user submits a report, the server validates it, the database stores it, and an authorized administrator can update the status. Additional goals include making error messages understandable, keeping records consistent, and structuring the code so that future features can be added.

3. PROBLEM STATEMENT, OBJECTIVES AND SCOPE (CONTINUED)

3.4 Scope of the Project

The project is intended for a web environment and can be demonstrated using a small set of sample reports. The initial implementation should prioritize reliable report creation, retrieval, and status updates before adding advanced automation.

Included Scope

- Create and view waste reports.
- Record waste category, location, description, image URL, status, and timestamps.
- Allow authorized status updates.
- Provide an administrator view of reports.
- Search and filter reports by supported fields.
- Use MongoDB as the central data store.
- Provide responsive pages for desktop and mobile devices.
- Implement authentication and roles if required by the project.

Out of Scope for the Basic Version

- Automatic physical collection of waste.
- Guaranteed response times from municipal services.
- Automatic identification of waste from images unless an AI model is added.
- Live vehicle tracking unless GPS tracking is integrated.
- Sensor-based bin monitoring unless hardware is installed.
- Automatic routing of collection trucks unless a routing module is developed.

3.5 Assumptions and Limitations

The system assumes users provide reasonably accurate information and that administrators or collection teams review reports. Internet connectivity is required. GPS and map features depend on browser permissions and external services. Uploaded images require file validation and storage configuration. The application can organize information, but the actual removal of waste remains a physical operational responsibility.

4. FEATURES AND FUNCTIONAL REQUIREMENTS

4.1 Waste Reporting

The report form is the primary entry point to the system. It should ask the user for only the information needed to understand and act on the issue. Typical fields include a short title, waste type, location, description, and optional image. Required fields should be clearly identified, and validation errors should explain what needs to be corrected.

4.2 Waste Categories

A predefined list of categories makes reports easier to compare and filter. Possible categories include household waste, plastic waste, organic waste, electronic waste, construction waste, hazardous waste, and mixed waste. Hazardous-waste reports may require additional handling guidance and should be routed only to suitably authorized personnel.

4.3 Image Upload

A photograph can help an administrator understand the type and scale of the problem. The system should restrict allowed file types and maximum file size. For a typical web application, the image can be uploaded to a dedicated storage provider and its URL stored with the report. The application should not trust a file solely because its extension appears valid.

4.4 Location Management

Location information can be entered as an address, landmark, or area name. Latitude and longitude may be stored when valid coordinates are available. In a later version, the user can grant permission for browser-based location detection and confirm the selected point on a map.

4.5 Status Tracking

Status	Meaning	Typical action
Pending	A report has been received and awaits action.	Review and prioritize.
In Progress	The issue is being addressed.	Coordinate collection or cleanup.
Resolved	The reported issue has been addressed.	Close the report and retain its history.

4. FEATURES AND FUNCTIONAL REQUIREMENTS (CONTINUED)

4.6 Administrator Dashboard

The dashboard should provide a clear view of current workload. Useful summary cards include total reports, pending reports, reports in progress, and resolved reports. Administrators should be able to open a record, review its information, update its status, and search for reports. Dashboard totals should be calculated from actual database records.

4.7 Search and Filtering

Filtering can reduce the time required to find relevant reports. Common filters include status, waste category, location, and date range. For large datasets, pagination should be used so the application does not load every record at once.

4.8 Functional Requirements

ID	Requirement	Description
FR-01	Create report	Submit a new waste report.
FR-02	View reports	Retrieve records allowed by the user's role.
FR-03	Upload image	Attach a supported image to a report.
FR-04	Store location	Save address and optional coordinates.
FR-05	Select waste type	Choose a category from an approved list.
FR-06	Update status	Allow authorized administrators to update status.
FR-07	Delete report	Permit deletion only to authorized roles, if enabled.
FR-08	Search	Find reports using supported query terms.
FR-09	Filter	Filter by status, location, category, or date.
FR-10	Dashboard	Display summary counts based on saved reports.
FR-11	Authentication	Register and log in if implemented.
FR-12	Authorization	Reject protected actions from unauthorized users.

4.9 Non-Functional Requirements

Performance: common operations should respond promptly. Security: inputs and protected actions must be validated. Reliability: errors should be handled without corrupting existing data. Usability: forms and status labels should be easy to understand. Maintainability: code should be modular. Scalability: the design should support larger datasets and future integrations.

5. ALGORITHMS AND METHODOLOGY

5.1 Development Methodology

The project can be developed incrementally. Begin with requirements and database fields, then create the backend API, connect the database, build the React screens, and integrate the complete workflow. Testing should be performed throughout development rather than postponed until the end.

11. Requirement analysis: define users, report fields, and permissions.
12. System design: define the architecture, data model, and API routes.
13. Backend development: implement validation and CRUD operations.
14. Frontend development: create forms, lists, details, and dashboards.
15. Integration: connect frontend requests to backend routes.
16. Testing: verify normal, invalid, and unauthorized requests.
17. Deployment: configure hosting, environment variables, and database access.

5.2 Three-Layer Architecture

Layer	Technology	Responsibilities
Presentation	React.js	Forms, report list, dashboards, status display, API calls.
Application	Node.js and Express.js	Routes, validation, business rules, authorization, database access.
Data	MongoDB	Report documents, location, categories, image URL, status, timestamps.

Architecture flow: User Interface → React.js → Express API running on Node.js → MongoDB. The API returns a response to the frontend, which updates the page to show the result.

5.3 Data Flow

18. The user enters report details in the React interface.
19. The frontend checks required fields and sends an HTTP request.
20. The Express route validates the request again on the server.
21. The backend processes an optional image upload.
22. The report document is stored in MongoDB.
23. The server returns a success or error response.
24. The React interface displays confirmation or actionable feedback.

5. ALGORITHMS AND METHODOLOGY (CONTINUED)

5.4 Report Submission Algorithm

Purpose: validate a submitted report and save it to the database.

```
START
Read location, waste_type, description, and optional image
Validate required fields
IF required fields are invalid
    Return validation error
ELSE
    Process image upload if enabled
    Create report with status = "Pending"
    Save report in MongoDB
    IF save succeeds
        Return success response
    ELSE
        Return error response
    END IF
END IF
STOP
```

5.5 Status Update Algorithm

A status update must be restricted to an authorized administrator. The server should validate the status value and verify that the report exists before saving changes.

```
START
Receive report ID and new status
Verify administrator authorization
IF authorization fails
    Return unauthorized response
ELSE
    Find report in MongoDB
    IF report does not exist
        Return not-found response
    ELSE IF status is not allowed
        Return validation error
    ELSE
        Update status and updatedAt
        Save and return updated report
    END IF
END IF
STOP
```

5.6 Filtering Algorithm

25. Receive supported filter parameters.
26. Validate the parameters and normalize their format.
27. Construct a database query from approved fields.
28. Retrieve matching reports using pagination.
29. Return results and pagination metadata to the frontend.

5.7 Status History

The basic version may store only the current status and the latest update time. A more robust version can maintain a separate history containing the previous status, new status, timestamp, and administrator ID. This history helps explain when the report changed state and supports accountability.

6. TECHNOLOGIES AND IMPLEMENTATION

6.1 MERN Stack

Technology	Role in the Project
MongoDB	Stores report documents and associated metadata.
Express.js	Provides HTTP routing and middleware.
React.js	Builds reusable, interactive user-interface components.
Node.js	Runs the backend JavaScript application.
HTML5	Provides page structure.
CSS3	Controls layout, typography, and responsive styling.
JavaScript	Implements browser and server-side behavior.
Cloudinary (optional)	Stores uploaded images and returns accessible URLs.
Git and GitHub	Track source-code changes and support collaboration.
Postman	Sends requests to test backend API behavior.

6.2 Why the MERN Stack?

The MERN stack allows JavaScript to be used across the frontend and backend. React supports component-based interfaces, Express provides a lightweight routing layer, Node.js handles asynchronous server operations, and MongoDB stores flexible document records. This combination is suitable for a project where report fields may evolve as new features are added.

6.3 Frontend Structure

The frontend can be organized into components such as NavigationBar, ReportForm, ReportList, ReportCard, ReportDetails, StatusBadge, SearchFilters, and AdminDashboard. Shared components improve consistency and make it easier to update the design.

6.4 Backend Structure

The backend can separate route definitions, controllers, database models, middleware, and configuration. This separation keeps request handling understandable and reduces duplication. Error-handling middleware should return consistent status codes and avoid revealing internal stack traces to end users.

6. TECHNOLOGIES AND IMPLEMENTATION (CONTINUED)

6.5 Proposed REST API Endpoints

HTTP Method	Endpoint	Purpose
POST	/api/reports	Create a report.
GET	/api/reports	List authorized reports; support filters and pagination.
GET	/api/reports/:id	Retrieve one report.
PUT	/api/reports/:id	Update permitted report fields.
PATCH	/api/reports/:id/status	Update report status.
DELETE	/api/reports/:id	Delete a report when permitted.

These endpoints are a proposed design and must be adjusted to match the routes implemented in the application.

6.6 Database Design

A MongoDB collection named reports can store each report as a document. The document should use consistent field names and controlled status values. MongoDB automatically provides an identifier for each document.

```
{
  "_id": "generated-mongodb-id",
  "title": "Garbage accumulation near park",
  "wasteType": "Household Waste",
  "location": {
    "address": "Near Main Road",
    "latitude": 17.0,
    "longitude": 78.0
  },
  "description": "Garbage has accumulated near the park entrance.",
  "imageUrl": "https://example.com/waste-image.jpg",
  "status": "Pending",
  "createdAt": "2026-10-09T10:00:00.000Z",
  "updatedAt": "2026-10-09T10:00:00.000Z"
}
```

The record is illustrative only. Replace the address, coordinates, image URL, and timestamps with real values in a working application. A userId field can link the report to its creator when authentication is implemented.

6.7 Data Validation

- Ensure required text fields are not empty.
- Restrict wasteType and status to approved values.
- Validate the report identifier before querying.
- Validate coordinate ranges if coordinates are accepted.
- Set timestamps on the server rather than trusting client values.
- Limit image size and accept only approved file types.

. TECHNOLOGIES AND IMPLEMENTATION (CONTINUED)

6.8 Implementation Workflow

30. Create frontend and backend project folders.
31. Configure environment variables and dependencies.
32. Create the MongoDB connection and report model.
33. Implement report creation and retrieval routes.
34. Implement status updates and authorization.
35. Build React forms and report-list components.
36. Connect frontend actions to backend endpoints.
37. Add image storage if required.
38. Run tests and fix defects.
39. Deploy the application and verify the production configuration.

6.9 Image Management

Images should be treated as uploaded files, not trusted text. The application should enforce file size limits, check accepted content types, and use secure storage. The image URL can be stored in the report document. If the application allows public access to images, the team should consider whether photographs may expose personal or sensitive information.

6.10 Security Considerations

- Authenticate users before allowing access to protected features.
- Use role-based authorization for administrative operations.
- Hash passwords using a suitable password-hashing algorithm if accounts are implemented.
- Keep database credentials and service keys in environment variables.
- Validate and sanitize incoming data on the server.
- Use HTTPS for deployed applications.
- Apply rate limits and suitable CORS settings.
- Return clear errors without disclosing secrets or stack traces.
- Record important administrative changes when auditability is needed.

6.11 Deployment Considerations

The frontend and backend may be hosted separately or together depending on the chosen platform. The deployed backend needs a secure database connection string and any image-storage credentials. Production configuration should not be committed to a public repository. Before a demonstration, test the deployed URL, database connection, report submission, and status update workflow.

7. RESULTS, TESTING AND PERFORMANCE

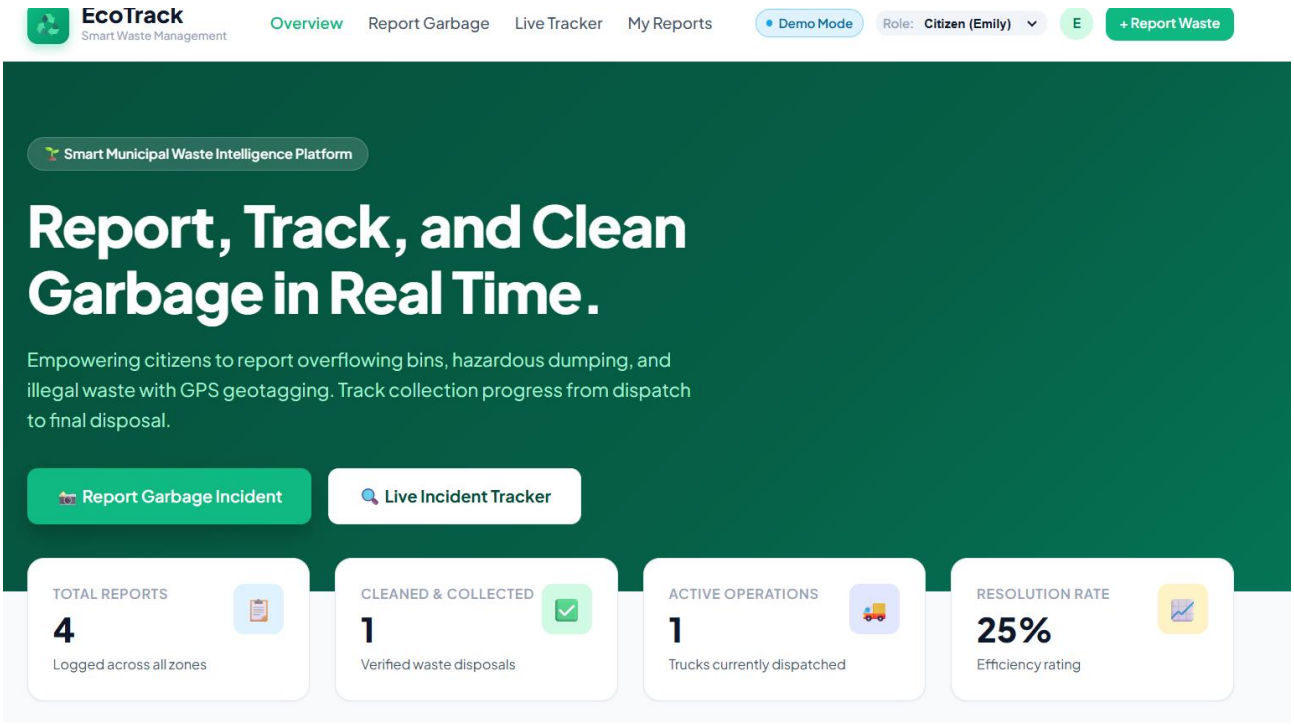
7.1 Expected Results

The expected result is a web application in which a citizen can submit a waste report and an authorized administrator can view and update it. The report should persist in the database and retain its current status when the page is refreshed. Search and filtering should return records that match the selected criteria.

- Successful creation of waste reports.
- Persistent storage of report details in MongoDB.
- Display of report information in the React interface.
- Status updates through protected backend routes.
- Search and filtering based on supported fields.
- Dashboard totals calculated from database records.

These are expected outcomes, not claims that the features have already been verified. Confirm each outcome using the working application.

7.2 Screenshots to Include



7.3 Testing Approach

Testing should cover individual functions, API behavior, database integration, user workflows, invalid inputs, and access control. Manual testing can be performed through the browser and Postman. Automated tests may be added for validation functions, API routes, and status transitions.

7. RESULTS, TESTING AND PERFORMANCE (CONTINUED)

7.4 Functional Test Cases

ID	Test Scenario	Expected Result	Actual Result
TC-01	Submit a valid report	Report is saved successfully	To be tested
TC-02	Omit a required field	Validation error is shown	To be tested
TC-03	Retrieve report list	Authorized reports are returned	To be tested
TC-04	Request an invalid report ID	Not-found or validation response	To be tested
TC-05	Authorized status update	Status changes successfully	To be tested
TC-06	Unauthorized status update	Request is rejected	To be tested
TC-07	Upload disallowed image	Upload is rejected	To be tested
TC-08	Filter by Pending status	Only matching records are shown	To be tested
TC-09	Delete without permission	Deletion is rejected	To be tested
TC-10	Database connection failure	Error handled without application crash	To be tested

7.5 Integration Testing

Integration testing verifies that React sends the correct request, Express processes it, MongoDB stores or retrieves the expected document, and the response is displayed correctly. Test the complete report lifecycle rather than testing each screen in isolation.

7.6 User Acceptance Testing

A small group of test users can be asked to submit a sample report, find it in the dashboard, and check its status. Feedback should be collected about clarity of the form, ease of locating reports, usefulness of status labels, and readability on mobile screens.

7.7 Defect Recording

When a test fails, record the test ID, steps to reproduce, expected result, actual result, severity, and resolution. Retest the affected workflow after the fix to confirm that the defect has been addressed.

7. RESULTS, TESTING AND PERFORMANCE (CONTINUED)

7.8 Performance Evaluation

Performance should be measured on the running application rather than estimated in the report. Useful metrics include average API response time, report submission time, report-list retrieval time, error rate, image upload time, and database query duration.

A simple experiment can use sample datasets of 100, 1,000, and 10,000 reports. For each dataset, measure the time required to retrieve the first page of reports and filter by status. Run each measurement several times and report the average, while recording the test environment and network conditions.

7.9 Performance Improvement Techniques

- Add indexes to fields used frequently in filters.
- Use pagination instead of returning every report.
- Limit returned fields when full documents are unnecessary.
- Resize large images and use suitable compression.
- Use efficient queries and avoid repeated database calls.
- Keep React components focused and avoid unnecessary re-renders.
- Cache only data that can safely tolerate being temporarily stale.

7.10 Results Summary Template

Metric	Measured Result	Notes
Total test cases	To be measured	Count executed test cases.
Passed test cases	To be measured	Record only verified passes.
Failed test cases	To be measured	Document defects.
Average API response time	To be measured	Record units and environment.
Report submission success rate	To be measured	Use repeated test submissions.
Status update success rate	To be measured	Test authorized and unauthorized requests.
Dataset size	To be selected	Record the number of sample reports.

Do not replace these placeholders with invented measurements. Enter the actual values after running tests.

8. CONCLUSION, FUTURE SCOPE AND REFERENCES

8.1 Conclusion

The Smart Waste Management System proposes a digital method for reporting and tracking garbage collection activities. By providing structured report submission, centralized data storage, and status updates, the application can make waste-related complaints easier to organize and follow.

The MERN stack supports this design through React-based interfaces, Express APIs, Node.js backend processing, and MongoDB document storage. The project provides an opportunity to apply full-stack development concepts to a practical environmental problem, including form validation, CRUD operations, database design, access control, image management, and testing.

The application does not independently collect or dispose of waste. Its practical value depends on accurate reports, secure and reliable software, use by citizens and administrators, and coordination with the people responsible for collection. Once implemented and tested, the system can serve as a foundation for additional smart-city features.

8.2 Benefits

- Centralizes report information in a searchable database.
- Provides a visible status for each complaint.
- Allows image and location details to accompany reports.
- Helps administrators review unresolved issues.
- Creates data that can support future analysis.
- Uses widely adopted web technologies and modular architecture.

8.3 Limitations

- The software cannot guarantee when a physical issue will be resolved.
- Users may submit inaccurate, incomplete, or duplicate reports.
- Image and GPS features depend on configuration and permissions.
- Performance and reliability must be verified under realistic conditions.

8. CONCLUSION, FUTURE SCOPE AND REFERENCES (CONTINUED)

8.4 Future Scope

GPS-based location detection: Capture a user's location with permission and allow the user to confirm the selected point.

Interactive maps: Display reported locations on a map and support geographic filtering.

Real-time notifications: Send updates when a report is accepted, assigned, or resolved.

AI-based waste classification: Explore image classification to suggest a waste category from a photograph.

Duplicate report detection: Flag reports with similar locations, categories, and descriptions for review.

Smart-bin integration: Connect bin sensors that report fill levels or equipment faults.

Collection route optimization: Recommend collection routes based on locations and operational constraints.

Analytics and visualization: Display trends by category, area, and resolution time.

Mobile application: Provide a dedicated mobile experience for field users and citizens.

Multilingual support: Offer regional-language interfaces and accessible content.

QR-code reporting: Allow users to scan a code attached to a public bin to start a report.

Municipal integration: Connect to authorized municipal systems where an approved integration exists.

8.5 Suggested Development Priorities

40. First, complete reliable report creation, viewing, and status updates.
41. Second, implement authentication and role-based permissions.
42. Third, add image uploads and location validation.
43. Fourth, improve the dashboard with search, filters, and pagination.
44. Finally, consider maps, notifications, analytics, and AI features based on available time and resources.

8. CONCLUSION, FUTURE SCOPE AND REFERENCES (CONTINUED)

8.6 References

The following official documentation sources are useful for implementing and verifying the technologies discussed in this report:

1. MongoDB Documentation. Database design, queries, indexes, and deployment. Available at: https://www.mongodb.com/docs/

2. Express.js Documentation. Routing, middleware, and HTTP request handling. Available at: https://expressjs.com/

3. React Documentation. Components, state, events, and frontend design. Available at: https://react.dev/

4. Node.js Documentation. Runtime APIs and server-side JavaScript. Available at: https://nodejs.org/docs/

5. MDN Web Docs. HTML, CSS, JavaScript, forms, and web APIs. Available at: https://developer.mozilla.org/

6. Postman Learning Center. API requests and testing workflows. Available at: https://learning.postman.com/

7. Cloudinary Documentation. Image upload and media management. Available at: https://cloudinary.com/documentation

8. Git Documentation. Version control and collaboration. Available at: https://git-scm.com/doc

8.7 Reference Usage Note

Use the official documentation for the versions and services actually used in the implementation. If the project uses additional libraries, hosting platforms, map providers, or authentication services, add their official documentation to this list.

APPENDIX A. PROPOSED PROJECT MODULES

Module	Responsibility	Example output
User module	Manages accounts and roles if implemented.	Authenticated session
Waste reporting	Creates reports with structured fields.	New report record
Image upload	Validates and stores image files.	Image URL
Location module	Stores address and optional coordinates.	Location object
Report management	Retrieves and updates report records.	Report list/details
Status tracking	Maintains current status and timestamps.	Updated status
Admin dashboard	Summarizes report counts and workload.	Dashboard cards
Search module	Filters reports by supported criteria.	Filtered results

APPENDIX B. PROPOSED PROJECT WORKFLOW

45. A citizen notices garbage accumulation.
46. The citizen opens the Smart Waste Management website.
47. The citizen enters the location, category, description, and optional image.
48. The frontend sends the report to the backend.
49. The backend validates the request and stores the record in MongoDB.
50. An authorized administrator reviews the report.
51. The administrator updates the status as collection progresses.
52. The user checks the current status.
53. The report is marked Resolved after the issue has been addressed.

APPENDIX C. GLOSSARY AND SUBMISSION CHECKLIST

Term	Meaning
MERN	MongoDB, Express.js, React.js, and Node.js.
CRUD	Create, Read, Update, and Delete operations.
API	Application Programming Interface; a defined way for software components to communicate.
REST API	An HTTP-based interface that exposes resources through methods such as GET, POST, PUT, PATCH, and DELETE.
MongoDB	A document-oriented database that stores records in BSON documents.
React.js	A JavaScript library for building user interfaces.
Node.js	A JavaScript runtime used to run server-side code.
Express.js	A web framework for Node.js.
GPS	Global Positioning System, which can provide geographic coordinates.
HTTP status code	A numeric response code indicating the outcome of an HTTP request.

Final Submission Checklist

- Enter the team name, member names, roll numbers, and faculty guide.
- Confirm that features and endpoints in the report match the actual application.
- Insert screenshots from the working website with figure captions.
- Run the test cases and enter real results.
- Measure performance if performance claims are included.
- Remove features from the report if they were not implemented or clearly label them as future scope.
- Check spelling, page layout, and reference links before submission.

End of Report